

计划与求解提示：提升大语言模型的零样本思维链推理能力

Lei Wang¹ Wanyu Xu² Yihuai Lan³ Zhiqiang Hu³

Yunshi Lan⁴ Roy Ka-Wei Lee³ Ee-Peng Lim^{1*}

¹新加坡管理大学

²西南交通大学

³新加坡科技设计大学

⁴华东师范大学

摘要

近期，大语言模型（Large Language Models, LLMs）在各种自然语言处理（Natural Language Processing, NLP）任务中展现出了令人瞩目的性能。为了解决多步推理任务，少样本思维链（Chain-of-Thought, CoT）提示包含了一些人工精心设计的逐步推理示例，这些示例使得大型语言模型能够显式地生成推理步骤，从而提高其推理任务的准确率。为了消除这种人工成本，零样本思维链（Zero-shot-CoT）将目标问题描述与“让我们一步一步地思考（Let's think step by step）”拼接起来，作为输入提示词提供给大型语言模型。尽管零样本思维链取得了成功，但它仍然存在三个缺陷：计算错误、步骤缺失错误以及语义理解错误。为了解决步骤缺失错误，我们提出了计划与求解（Plan-and-Solve, PS）提示法。该方法由两个部分组成：首先，制定一个计划将整个任务划分为更小的子任务；然后，根据该计划执行这些子任务。为了解决计算错误并提高生成推理步骤的质量，我们利用更详细的指令对计划与求解（PS）提示法进行了扩展，从而衍生出 PS+ 提示法。我们在涵盖三种推理问题的十个数据集上评估了我们提出的提示策略。在 GPT-3 上的实验结果表明，我们提出的零样本提示法在所有数据集上均以较大优势稳定优于零样本思维链（Zero-shot-CoT），其表现与零样本程序思维（Zero-shot-Program-of-Thought, PoT）提示法相当或更优，并且在数学推理问题上具有与 8 样本思维链（8-shot CoT）提示法相当的性能。<https://github.com/AGI-Edgerunners/Plan-and-Solve-Prompting>。

1 引言

近期，大语言模型 (Brown et al. 2020; Thoppilan et al. 2022; Chowdhery et al. 2023) 在各种自然语言处理 (Natural Language Processing, NLP) 任务中证明了其极高的有效性。与以往的预训练语言模型 (Pre-trained Language Models, PTMs) (Devlin et al. 2019; Liu et al. 2019) 不同，出于商业考量和潜在的滥用风险，这些 LLMs 通常以服务的形式提供，用户无法直接访问模型参数 (Sun et al. 2022)。因此，针对下游任务对 LLMs 进行微调 (fine-tune) 是一项具有挑战性的工作 (He et

al. 2021; Housby et al. 2019; Devlin et al. 2019)。相反，我们利用 LLMs 来解决复杂的推理问题，其方法是通过指令（或触发句）来激发它们基于其内在知识的强大推理能力。迄今为止，LLMs 已经展现出了令人瞩目的能力：只需为其提供少量解释性示例（即少样本学习，few-shot learning），或提供无需示例即可解决新问题的提示（即零样本学习，zero-shot learning），它们就能解决新的推理问题。

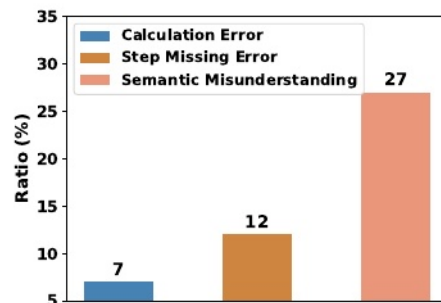


图 1 使用 GPT-3 大语言模型时，Zero-shot-CoT 返回错误答案的 46 个 GSM8K 问题的错误分析。遵循 Wei, Wang, et al. (2022) 和 Z. Wang et al. (2023) 的做法，我们将每个错误答案分别归类为“计算错误”（7%）、“步骤缺失错误”（12%）或“语义理解错误”（27%）。

为了使用 LLMs 处理多步复杂的推理任务，Wei 等人 (2022b) 提出了少样本思维链（Chain-of-Thought, CoT）提示法，该方法利用少量人工设计的逐步推理演示示例，使 LLMs 能够在预测最终答案之前显式地生成中间推理步骤。在 Kojima et al. (2022) 的研究中，零样本思维链（Zero-shot-CoT）通过将“让我们一步一步地思考（Let's think step by step）”附加到输入给 GPT-3 等 LLMs 的目标问题之后，消除了提示中对人工设计示例的需求。这种简单的提示策略出人意料地使 LLMs 能够产生与少样本 CoT 提示法相似的性能。

尽管 Zero-shot-CoT 在解决多步推理任务方面取得了显著成功，但在抽取 100 个算术测试样本的分析中，其结果依然暴露出三个缺陷（如图 1 所示）：(i) 计算错误（在 7% 的测试样本中）：计算过程中的失误导致了错误答案；(ii) 步骤缺失错误（在 12% 的测试样本中）：某些中间推理步骤被遗漏，这在涉及大量步骤的问题中尤为常见；(iii) 语义理解错误（在 27% 的测试样本中）：存在对问题语义理解不当及推理步骤连贯性不足等其他错误，这很可能是由于 LLMs 的能力不足所致。

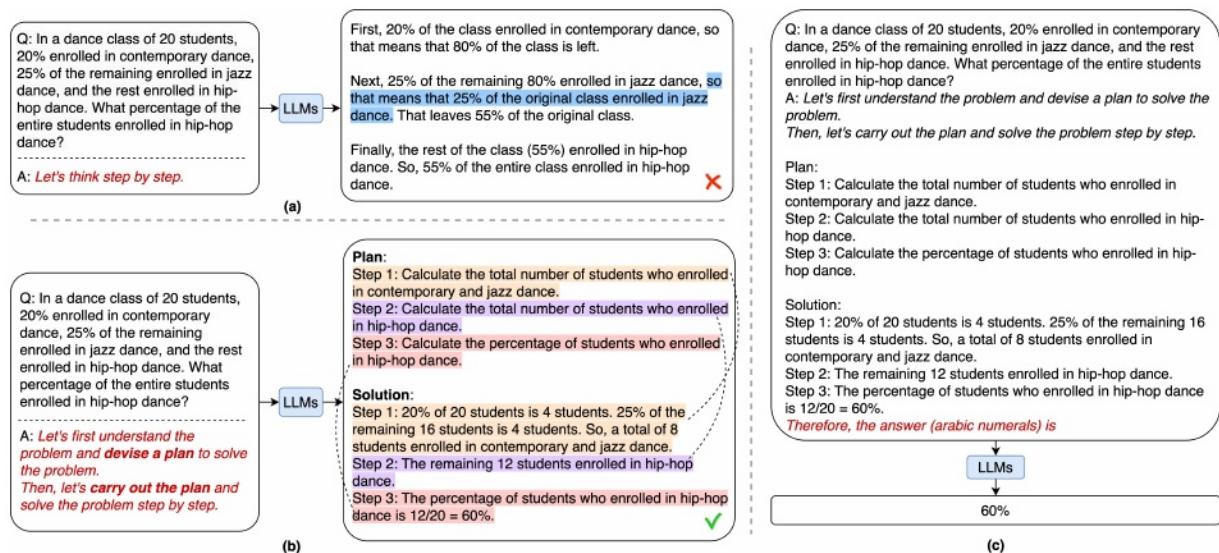


图2 使用 (a) 零样本思维链 (Zero-shot-CoT) 提示法、(b) 计划与求解 (PS) 提示法和 (c) 答案提取提示法时，GPT-3 的输入和输出示例。虽然 Zero-shot-CoT 通过“让我们一步一步地思考”来鼓励 LLMs 生成多步推理，但当问题复杂时，它仍可能生成错误的推理步骤。与 Zero-shot-CoT 不同，PS 提示法首先要求 LLMs 制定一个解决问题的计划（即生成一个逐步的计划），并执行该计划以求得答案。

为了解决 Zero-shot-CoT 因缺失推理步骤而引发的问题，我们提出了计划与求解（Plan-and-Solve, PS）提示法。它由两部分组成：首先，制定一个计划，将整个任务划分为更小的子任务，然后根据该计划执行这些子任务。在我们的实验中，我们仅仅将 Zero-shot-CoT 中的“让我们一步一步地思考”替换为“让我们首先理解问题，并制定一个计划来解决该问题。然后，让我们执行该计划，一步一步地解决问题”（见图 2 (b)）。

为了解决 Zero-shot-CoT 的计算错误并提升生成推理步骤的质量，我们在 PS 提示法中加入了更详细的指令。具体而言，我们增加了“提取相关变量及其对应的数字”和“计算中间结果（注意计算过程与常识）”这两项指令对其进行了扩展。这种提示的变体被称为 PS+ 提示策略（见图 3 (b)）。尽管它十分简单，但 PS+ 策略极大地提高了生成的推理过程的质量。此外，这种提示策略可以轻松定制，以解决除数学推理之外的各类问题，例如常识和符号推理问题。

我们在六个数学推理数据集上评估了我们提出的提示方法，包括 AQUA (Ling et al. 2017)、GSM8K (Cobbe et al. 2021)、MultiArith、AddSub、SingleEq 和 SVAMP (Patel et al. 2021)，两个常识推理数据集（CommonsenseQA (Talmor et al. 2019) 和 StrategyQA (Geva et al. 2021)），以及两个符号推理数据集（Last Letter 和 Coin Flip (Wei, Wang, et al. 2022)）。使用 GPT-3 进行的实验结果表明，我们提出的零样本 PS+ (Zero-shot-PS+) 提示法在所有推理问题和数据集上均以较大优势稳定优于 Zero-shot-CoT，并且与零样本程序思维（Program-of-Thought, PoT）提示法 (Chen et al. 2022) 表现相当或更好。此外，虽然 PS+ 提示不需要人工演示示例，但它在算术推理中的表现与 8 样本（8-shot）CoT 提示法相仿。

总而言之，我们的结果表明：(a) 零样本 PS 提示法能够生成比 Zero-shot-CoT 提示法更高质量的推理过程，这是因为 PS 提示提供了更详细的指令，指导 LLMs 执行正确的推理任务；(b) 零样本 PS+ 提示法在某些数据集上的表现超越了少样本（Few-shot）人工 CoT 提示法，表明它在某些情况下具备超越人工少样

本 CoT 提示法的潜力，这有望激发学术界后续开发出更多的新型 CoT 提示方法，以有效激发 LLMs 的推理能力。

2 计划与解决提示

概述。我们引入了 PS 提示法，这是一种新型的 Zero-shot-CoT 提示方法，它使 LLMs 能够在预测输入问题的最终答案之前，显式地设计一个解决给定问题的计划，并生成中间推理过程。与包含逐步少样本演示示例的先前少样本 CoT 提示方法不同，零样本 PS 提示方法不需要演示示例，其提示词仅包含问题本身和一个简单的触发句。与 Zero-shot-CoT 相似，零样本 PS 提示法包含两个步骤。在步骤 1 中，提示词首先使用所提出的提示模板进行推理，以生成推理过程和问题的答案。在步骤 2 中，它通过使用答案提取提示（例如“因此，答案（阿拉伯数字）是”）来提取最终答案以供评估。

2.1 步骤 1：生成推理过程的提示

为了解决输入问题，同时避免因计算不正确和推理步骤缺失而导致的错误，此步骤旨在构建满足以下两个标准的模板：

- 模板应诱导 LLMs 确定子任务并完成这些子任务。
- 模板应引导 LLMs 更加关注计算和中间结果，并尽可能确保它们的执行是正确的。

为了满足第一个标准，我们遵循 Zero-shot-CoT 的做法，首先使用一个简单的模板“问：[X]。答：[T]”将输入数据样本转换为提示词。具体而言，输入位 [X] 包含输入的问题陈述，并在输入位 [T] 中指定了一个手工构建的指令，以触发 LLMs 生成包含计划及完成该计划所需步骤的推理过程。

在 Zero-shot-CoT 中，输入位 [T] 中的指令包含了触发指令“让我们一步一步地思考”。而我们的零样本 PS 提示法则包含了“制定一个计划”和“执行该计划”的指令，如图 2 (b) 所示。因此，提示词将是“问：[X]。答：让我们首先理解问题并制定一个计划来解决该问题。然后，让我们执行该计划，一步一步地解决问题。

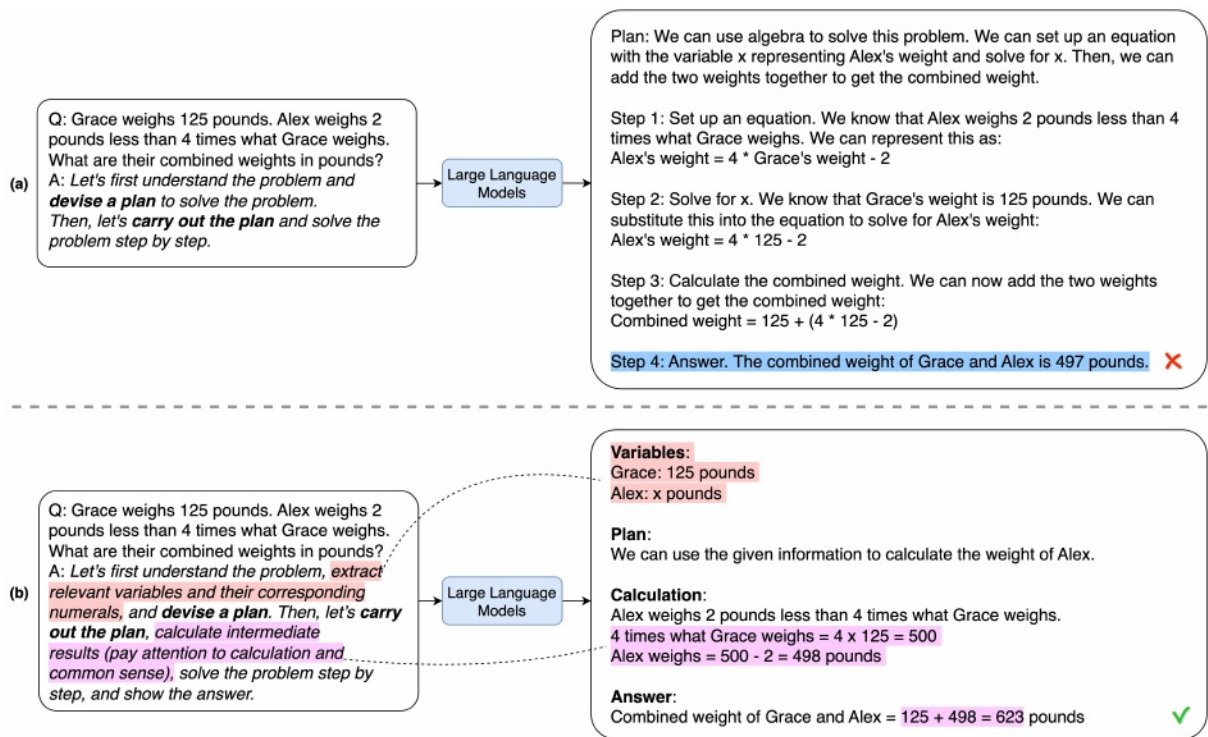


图 3 使用 (a) 计划与求解 (PS) 提示法和 (b) 带有更详细指令的计划与求解提示法 (PS+ 提示法) 时, GPT-3 的输入和输出示例。PS+ 提示法极大地提高了生成的推理过程的质量。

随后, 我们将上述提示词传递给 LLM, 后者随即输出相应的推理过程。与 Zero-shot-CoT 保持一致, 我们的方法在默认情况下使用贪心解码 (greedy decoding) 策略 (1 个输出链) 来生成输出。

为了满足第二个标准, 我们使用更详细的指令对基于计划的触发句进行了扩展。具体来说, 在触发句中添加了“注意计算”, 以要求 LLMs 尽可能准确地执行计算。为了减少因遗漏必要推理步骤而导致的错误, 我们在提示中加入了“提取相关变量及其对应的数字”, 显式地指示 LLMs 不要忽略输入问题陈述中的相关信息。我们假设如果 LLMs 遗漏了相关且重要的变量, 就更有可能缺失相关的推理步骤。如图 5 所示, 对生成的变量内容和推理步骤缺失错误进行的相关性分析 (相关性值小于 0), 从经验上支持了这一假设。

2.2 步骤 2: 答案提取提示

与 Zero-shot-CoT 类似, 我们在步骤 2 中设计了另一个提示, 以促使 LLM 从步骤 1 中生成的推理文本中提取最终的数值答案。该提示将答案提取指令附加在第一个提示词之后, 紧接着是 LLM 生成的推理文本。通过这种方式, 期望 LLM 能够以所需的格式返回最终答案。

基于图 3 (b) 中的示例, 步骤 2 中使用的提示词将包含: “问: Grace 重 125 磅..... 变量: Grace: 125 磅..... 答案: Grace 和 Alex 的总重量 = 125 + 498 = 623 磅。因此, 答案 (阿拉伯数字) 是”。对于该示例, LLM 返回的最终答案为“623”。

3 实验设置

3.1 基准测试

所提出的方法在来自三类推理问题的十个基准

数据集上进行了评估: **算术推理:** (1) GSM8K 数据集 (Cobbe et al. 2021), 这是一个由人类问题编写者创建的高质量、语言多样化的小学数学应用题数据集; (2) SVAMP 基准测试 (Patel et al. 2021), 通过对另一个现有数据集中的一组问题进行简单修改, 构成了针对最高四年级学生的一元算术应用题; (3) MultiArith 数据集 (Roy and Roth 2015), 包含需要多个推理步骤和运算的数学应用题; (4) AddSub 数据集 (Hosseini et al. 2014), 包含加减法算术应用题; (5) AQUA 数据集 (Ling et al. 2017), 包含带有自然语言解释的代数应用题; (6) SingleEq 数据集 (Koncel-Kedziorski et al. 2016), 包含涉及非负有理数和单变量的多次数学运算的单方程小学代数应用题。**常识推理:** (7) CSQA 基准数据集 (Talmor et al. 2019), 包含需要不同类型的常识知识才能获得正确答案的多项选择题; (8) StrategyQA 基准数据集 (Geva et al. 2021), 其中的问题需要多步推理, 但并未给出推理步骤, 因此需要推断得出。**符号推理:** (9) 尾字母拼接 (Last Letter Concatenation) 数据集 (Wei, Wang, et al. 2022), 要求将姓名中单词的最后一个字母拼接起来 (例如, “James Brown” -> “sn”); (10) 抛硬币 (Coin Flip) 数据集 (Wei, Wang, et al. 2022), 包含根据问题中给出的步骤 (翻转或不翻转), 判断硬币是否仍然正面向上的问题。表 1 展示了数据集的统计信息。

表 1 所评估数据集的详细信息。Math: 算术推理。CS: 常识推理。Sym.: 符号推理。

Dataset	Domain	# Samples	Ave. words	Answer
MultiArith	Math	600	31.8	Number
AddSub	Math	395	31.5	Number
GSM8K	Math	1319	46.9	Number
AQUA	Math	254	51.9	Option
SingleEq	Math	508	27.4	Number
SVAMP	Math	1000	31.8	Number
CSQA	CS	1221	27.8	Option
StrategyQA	CS	2290	9.6	Yes / No
Last Letters	Sym.	500	15.0	String
Coin Flip	Sym.	500	37.0	Yes / No

表 2 在六个数学推理数据集上的准确率比较。最佳和次佳结果分别以粗体和下划线标出。

Setting	Method (text-davinci-003)	MultiArith	GSM8K	AddSub	AQuA	SingleEq	SVAMP	Average
Zero-Shot	CoT	83.8	56.4	85.3	38.9	88.1	69.9	70.4
	PoT	92.2	57.0	85.1	<u>43.9</u>	<u>91.7</u>	70.8	<u>73.5</u>
	PS (ours)	87.2	<u>58.2</u>	<u>88.1</u>	<u>42.5</u>	<u>89.2</u>	<u>72.0</u>	<u>72.9</u>
	PS+ (ours)	<u>91.8</u>	59.3	92.2	46.0	94.7	75.7	76.7
Few-Shot	Manual-CoT	93.6	58.4	91.6	48.4	93.5	80.3	77.6
	Auto-CoT	95.5	57.1	90.8	41.7	92.1	78.1	75.9

3.2 零样本与少样本基线方法

我们将提出的零样本 PS 和 PS+ 提示法与以下三种类型的提示基线方法进行了比较：(1) 零样本基线方法。我们引入了 Zero-shot-CoT (Kojima et al. 2022) 和零样本程序思维（Zero-shot-PoT）(Chen et al. 2022)。前者在没有任何演示示例的情况下，将“让我们一步一步地思考”附加到提示词中。后者使用 LLM（主要是 OpenAI Codex¹）生成一个 Python 程序，然后通过 Python 解释器上执行该生成的程序来得出答案；(2) 带有手动演示的少样本方法。手工思维链（Manual CoT）(Wei, Wang, et al. 2022)创建了八个手工编写的示例作为演示。(3) 带有自动演示的少样本方法。自动思维链（Auto-CoT）(Zhang et al. 2022) 通过保证多样性的聚类方法自动选择示例，并使用 Zero-shot-CoT 生成推理链，从而构建演示示例。

3.3 实施细节 (Implementations)

遵循自动思维链（Auto-CoT）(Zhang et al. 2022) 的设定，我们使用公开的 GPT-3 (Brown et al. 2020) (175B) 作为主干语言模型，它是拥有公开 API 的最广泛使用的 LLMs 之一。由于 text-davinci-003 是 text-davinci-002 的升级版，能够生成更高质量的文本，适应更复杂的指令，并且在较长篇幅的内容生成方面表现更好，因此我们在正文中报告了使用 GPT-3 的 text-davinci-003 引擎得出的结果。在我们所有的实验中，为了采用贪心解码策略，我们将温度设置为 0（即 argmax 采样）。我们还引入了两个少样本（few-shot）基线方法：手工思维链（Manual-CoT）和 Auto-CoT。正如原始论文 Wei et al. (2022)和 Zhang et al. (2022) 所建议的那样，我们为 MultiArith、GSM8K、AddSub、SingleEq 和 SVAMP 任务使用 8 个演示示例，为 AQUA 和 Last Letters 任务使用 4 个示例，为 CSQA 任务使用 7 个示例，为 StrategyQA 任务使用 6 个示例。在评估指标方面，我们遵循 Manual-CoT (Wei, Wang, et al. 2022) 的做法，并报告了所有方法在各个数据集上的准确率。

4 实验结果

4.1 主要结果

算术推理。表 2 报告了我们的方法与现有的零样本和少样本方法在算术推理数据集上的准确率对比。在零样本设定下，我们的 PS+ 提示法（即带有更详细指令的 PS 提示法）在所有算术推理数据集上均以较大优势稳定优于 Zero-shot-CoT。具体而言，除了 GSM8K 数据集提升了 2.9% 之外，PS+ 提示法在所有其他数据集上相较于 Zero-shot-CoT 的准确率至少提升了 5%。这一例外可能是由于 GSM8K 在语言复杂性方面是一个

更具挑战性的数据集。PS 提示法同样在所有数据集上优于 Zero-shot-CoT，其平均准确率比 Zero-shot-CoT 高出 2.5%。

与另一个具有竞争力的零样本基线方法 PoT 相比，PS(+) 和 PS 提示法的表现依然令人瞩目。在六个算术数据集中，PS+ 提示法在五个数据集上优于 PoT。PS 提示法也在三个算术数据集上优于 PoT。结果表明，在提示词中添加更详细的指令能够有效地激发 LLMs 产生更高质量的推理步骤。

与少样本方法 Manual-CoT 和 Auto-CoT 相比，PS+ 提示法的平均准确率（76.7%）略低于 Manual-CoT（77.6%），但高于 Auto-CoT（75.9%）。尽管这并非绝对公平的比较，但该结果表明零样本提示法能够超越少样本 CoT 提示法，这有望激发学术界进一步开发所需人工干预更少的新方法，以有效激发 LLMs 的推理能力。

常识推理。表 3 展示了在常识推理数据集（CommonsenseQA 和 StrategyQA）上的结果。在此比较中，我们仅包含了表现更好的零样本 PS+ 提示策略。由于零样本 PoT 无法处理此类问题，因此将其排除在外。虽然 PS+ 提示法在该问题上表现不如少样本 CoT（手工）（Few-Shot-CoT(Manual)），但它在 CommonsenseQA（71.9% 对 65.2%）和 StrategyQA（65.4% 对 63.8%）数据集上均稳定优于 Zero-shot-CoT。

表 3 常识性推理数据集上的准确率。

Method	CSQA	StrategyQA
Few-Shot-CoT (Manual)	78.3	71.2
Zero-shot-CoT	65.2	63.8
Zero-shot-PS+ (ours)	71.9	65.4

表 4 符号推理数据集上的准确率。

Method	Last Letter	Coin Flip
Few-Shot-CoT (Manual)	70.6	100.0
Zero-shot-CoT	64.8	96.8
Zero-shot-PS+ (ours)	75.2	99.6

符号推理。表 4 展示了在符号推理数据集（Last Letters 和 Coin Flip）上，PS+ 提示法与 Zero-shot-CoT 以及少样本 CoT（Few-shot-CoT）的准确率对比。零样本 PoT 同样被排除在外，因为它并非为该问题所设计。在 Last Letters 数据集上，我们的零样本 PS+ 提示法（75.2%）优于 Manual-CoT（70.6%）和 Zero-shot-CoT（65.2%）。在 Coin Flip 数据集上，零样本 PS+ 提示法（99.6%）略逊于 Manual-CoT（100.0%），但以较大优

表 5 在 GSM8K 和 SVAMP 数据集上测量的触发句性能比较（除编号 2 使用 code-davinci-002 外，其余均使用 text-davinci-003）。(*1) 表示 Zero-shot-CoT (Kojima et al. 2022) 中使用的触发句。(*2) 表示 Zero-shot-PoT (Chen et al. 2022) 中使用的触发句。

No.	Trigger Sentence		GSM8K	SVAMP
1	Let's think step by step.	(*1)	56.4	69.9
2	<pre>import math import numpy as np # Question: example['question'] # Answer this question by implementing a solver() function. def solver(): # Let's write a Python program step by step, and then return the answer # Firstly, we need define the following variable:</pre>	(*2)	57.0	70.8
3	Extract variables and assign their corresponding numerals to these variables first and then solve the problem step by step.		50.5	69.5
4	Firstly, extract variables and their corresponding numerals. Then, calculate intermediate variables. Finally, solve the problem step by step.		54.8	70.8
5	Let's first understand the problem and devise a plan to solve the problem. Then, let's carry out the plan and solve the problem step by step.		58.2	72.0
6	Let's first understand the problem, extract relevant variables and their corresponding numerals, and make a plan. Then, let's carry out the plan, calculate intermediate variables (pay attention to correct numerical calculation and commonsense), solve the problem step by step, and show the answer.		59.3	75.7

势优于 Zero-shot-CoT（96.8%）。更多实验结果示例可见附录 A.2。

4.2 分析

带有自治性提示的结果。自治性（Self-consistency, SC）(Wei, Wang, et al. 2022) 被提出用于通过生成 N 个推理结果并通过多数投票确定最终答案，从而减少 LLM 输出的随机性。使用 SC，通常期望各方法的结果更加一致且更好。因此，我们在 GSM8K 和 SVAMP 数据集上评估了带有 SC 的零样本 PS+ 提示法。我们将温度（temperature）设置为 0.7，并在包含 SC 的实验中 将 N 设置为 10。图 4 显示，在 GSM8K 和 SVAMP 上，带有 SC 的 PS+ 提示法（分别为 73.7% 和 84.4%）大幅 优于不带 SC 的情况（分别为 58.7% 和 75.7%）。前者 在 GSM8K 和 SVAMP 上也稳定优于带有 SC 的 Zero-shot-CoT（分别为 70.7% 和 81.7%），尽管 Zero-shot-CoT 同样受益于自治性方法而有所提升。

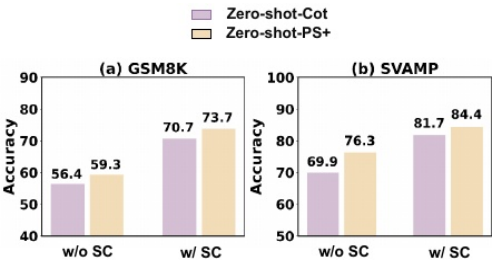


图 4 GSM8K 和 SVAMP 上具有和不具有自一致性(SC)的方法的结果

提示词的影响。表 5 展示了 6 种不同输入提示词 的性能比较。提示词 1 和 2 分别用于 Zero-shot-CoT 和 零样本 PoT。其余的提示词是在贪心解码下零样本 PS+ 提示策略步骤 1 中使用的提示变体。我们观察到，带 有变量和数字提取的提示词 3 的表现比 Zero-shot-CoT 的提示词 1 更差。原因在于提示词 3 没有包含设计 和完成计划的指令。然而，当我们添加更多关于中间 结果计算、计划设计和执行的指令时，零样本 PS+ 的 其他提示词表现良好。上述结果表明，当提示词包含 更详细的指令来指导 LLMs 时，LLMs 能够生成高质量 的

推理文本。针对不同推理问题的更多提示词可在附录 A.1 中找到。

错误分析 (Error Analysis)。为了定性评估零样本 PS+ 提示法对计算错误和推理步骤缺失错误的影响， 我们检查了 GSM8K 数据集上的错误分布。我们首先 从 GSM8K 中随机抽取 100 个问题，然后分别使用 Zero-Shot-CoT、零样本 PS 和零样本 PS+ 提示策略生 成推理文本并提取答案。Zero-Shot-CoT 在 46 个问题 上生成了错误的最终答案，零样本 PS 有 43 个，零样 本 PS+ 有 39 个。随后，我们分析并确定了所有这些问 题的错误类型，如表 6 所示。分析结果表明，PS+ 提 示法实现了最少的计算错误（5%）和步骤缺失错误 （7%），且语义理解错误与 Zero-shot-CoT 相当。零样 本 PS 的错误略多，但仍优于 Zero-shot-CoT。因此， 它们的计划与求解提示有效地引导了 LLMs 生成清晰 且完整的推理步骤。此外，PS+ 提示中增加的详细指 令（即“提取相关变量及其对应的数字”和“计算中间变 量”）使得 LLMs 能够生成高质量的推理步骤，从而进 一步减少计算错误。

表 6 在 GSM8K 数据集中抽取的 100 个样本上，Zero-shot-CoT、零 样本 PS (Zero-shot-PS) 提示法和零样本 PS+ 提示法得出错误最终答 案的错误类型分布。

Method	Calculation	Missing	Semantic
Zero-shot-CoT	7%	12%	27%
Zero-shot-PS	7%	10%	26%
Zero-shot-PS+	5%	7%	27%

生成的推理与错误类型的相关性分析。为了更深 入地了解 PS+ 提示法对错误类型的影响，我们检验了 生成的推理的各个子部分与错误类型之间的相关性。 具体而言，我们分析了生成的推理文本中变量定义、 推理计划和解决方案的存在情况，并将其与三种错误 类型关联起来。此分析研究所使用的问题集与早前错

误类型分析中使用的问题集相同。图 5 展示了变量定义、计划、解决方案的存在与三种不同类型错误之间的相关性矩阵。可以观察到，变量定义和计划的存在与计算错误和推理步骤缺失错误均呈负相关。零样本 PS+ 提示词能够通过减少计算错误和推理步骤缺失错误，进一步提升 LLMs 在数学推理问题上的性能。

探究 PS 预测中计划的存在性。为了确定 PS 做出的每个预测中是否都存在计划，我们随机抽取了 100 个数据样本并检查了它们相应的预测。我们的分析揭示，100 个预测中有 90 个确实包含了计划。这一观察结果表明，在如 GPT-3.5 和 GPT-4 等近期的 LLMs 中，已经涌现出了强大的规划能力。

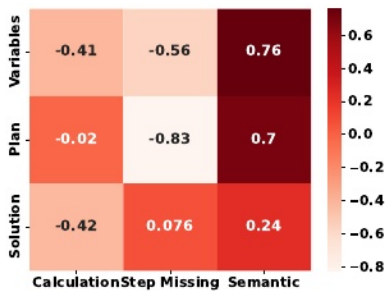


图 5 针对 Zero-shot-PS+ 提示法，从 GSM8K 数据集中随机抽样的 100 个数据样本的生成推理与错误类型的相关性分析。

5 相关工作

5.1 自然语言处理中的推理

众所周知，复杂的推理问题对 NLP 模型而言具有极大的挑战性，这类问题包括数学推理 (Cobbe et al. 2021; Patel et al. 2021; Ling et al. 2017; Koncel-Kedziorski et al. 2015)（需要理解数学概念、进行计算以及多步推理的能力）、常识推理 (Talmor et al. 2019; Geva et al. 2021)（需要基于常识知识进行判断的能力）以及逻辑推理 (Wei, Wang, et al. 2022)（需要通过应用形式逻辑规则来操作符号的能力）。在 LLMs 出现之前，Talmor et al. (2019) 曾利用微调 (fine-tuned) 后的 GPT 模型生成的解释来训练 NLP 模型，并发现训练后的模型在常识问答 (QA) 问题上取得了更好的性能。Hendrycks et al. (2021) 尝试使用带有标签的理由 (rationale) 来微调预训练语言模型，但发现这些微调后的模型难以轻松生成高质量的推理步骤。

Wei 等人 (2022a) 的近期工作表明，当参数规模扩展到数百亿时，LLMs 会展现出强大的推理能力，例如 GPT-3 (Brown et al. 2020) 和 PaLM (Chowdhery et al. 2023)。这些 LLMs 在提供少量演示样例的情况下，即可在各种不同的 NLP 任务中取得令人瞩目的性能。然而，这些模型在需要多步推理的问题上依然表现不佳。这可能是由于所提供的少数样例不足以彻底解锁 LLMs 的能力。

5.2 提示方法

为了发掘 LLMs 中的推理能力，Wei et al. (2022) 提出了 CoT 提示法，即在输入问题的最终答案之前附加多个推理步骤。凭借这种简单的少样本 (few-

shot) 提示策略，LLMs 能够在复杂的推理问题上取得显著的提升。随后，许多研究 (Wei, Tay, et al. 2022; Suzgun et al. 2023; Shaikh et al. 2023; Saparov and He 2022) 提出从不同方面进一步改进 CoT 提示法，其中包括提示词格式 (Chen et al. 2022)、提示词选择 (Lu et al. 2022)、提示词集成 (prompt ensemble) (Wei, Wang, et al. 2022; Li et al. 2023; Weng et al. 2022; Fu et al. 2022)、问题分解 (Zhou et al. 2022; Khot et al. 2022; Dua et al. 2022; Press et al. 2023) 以及规划 (planning) (Yao et al. 2022; Huang and Chang 2023; B. Wang et al. 2023; Liu et al. 2019; Sun et al. 2022)。Chen et al. (2022) 引入了 PoT 提示法，利用经过代码预训练的 LLMs 编写程序作为理由 (rationale)，从而将计算与推理分离开来。

为了免除人工成本，Kojima et al. (2022) 提出了 Zero-shot-CoT，在没有任何样例的情况下激发模型生成推理步骤。为了既利用演示示例的优势又最大限度地减少人工工作量，Zhang et al. (2022) 设计了 Auto-CoT。它首先通过对给定数据集进行聚类来自动获取 k 个具有多样性的示例。然后，它遵循 Zero-shot-CoT 的方式为所选出的示例生成理由。最后，通过将生成的理由添加到所选示例中作为 CoT 提示，从而构建出演示示例。

我们的工作与上述研究不同，侧重于以零样本 (zero-shot) 的方式激发 LLMs 进行多步推理。我们要求 LLMs 编写一个计划，将复杂的推理任务分解为多个较小的推理步骤。此外，我们在提示词中引入了详细的指令，以避免推理步骤中出现明显的错误。有关更多相关工作，请读者参考综述文章 (Huang and Chang 2023)。

6 结论

在本文中，我们发现 Zero-shot-CoT 仍然面临三个缺陷：计算错误、推理步骤缺失错误和语义理解错误。为了解决这些问题，我们引入了计划与求解提示策略 (PS 和 PS+ 提示法)。它们是新型的零样本提示方法，引导 LLMs 制定计划，将整个任务划分为更小的子任务，然后根据该计划执行这些子任务。在涵盖三种推理问题类型的十个数据集上的评估表明，PS+ 提示法优于先前的零样本基线方法，并且在多个算术推理数据集上表现出与少样本 CoT 提示法相当的性能。

总而言之，我们的结果表明：(a) 零样本 PS+ 提示法能够比 Zero-shot-CoT 提示法生成更高质量的推理过程，这是因为 PS 提示能够提供更详细的指令，以引导 LLMs 执行正确的推理；(b) 零样本 PS+ 提示法具有超越人工少样本 CoT 提示法的潜力，这有望激发后续开发出新的 CoT 提示方法，以诱导 LLMs 进行推理。此外，PS(+) 提示法是一个通用思想，可用于非推理任务，且对计划进行改进也是一个有趣的想法。我们将这些留待未来的工作去探索。

7 局限性

本工作存在两点局限性。首先，需要花费精力设计提示词以引导 LLMs 生成正确的推理步骤。GPT-3 模型对提示词中的表达方式很敏感，因此，我们需要精心设计提示词。其次，所提出的计划与求解提示法有助于解决计算错误和推理步骤缺失错误，但语义理解错误依然存在。在未来，我们将探索如何通过提示词而非升级 LLMs 来解决语义理解错误。

8 伦理

我们在六个数学推理数据集上进行了实验，包括 AQUA (Ling et al., 2017)、GSM8K (Cobbe et al., 2021)、MultiArith、AddSub、SingleEq 和 SVAMP (Patel et al., 2021)，两个常识推理任务 (CommonsenseQA (Talmor et al., 2019) 和 StrategyQA (Geva et al., 2021))，以及两个符号推理任务 (Last Letter 和 Coin Flip (Wei et al., 2022b))，其中 GSM8K 和 SVAMP 采用 MIT 许可证代码，AQUA 和 StrategyQA 采用 Apache-2.0 代码，其余数据集的许可证未指明。

本文提出的提示词不会收集或使用关于他人的个人信息。我们使用的提示词均已列在附录中，且本工作中的提示词不包含任何歧视特定个人或群体的词语。在本工作中，提示词也不会对他人的安全产生负面影响。

参考文献

Brown, Tom, Benjamin Mann, Nick Ryder, et al. 2020. "Language Models Are Few-Shot Learners." *Advances in Neural Information Processing Systems* 33: 1877–901.

Chen, Wenhui, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2022. "Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks." *arXiv Preprint arXiv:2211.12588*.

Chowdhery, Aakanksha, Sharan Narang, Jacob Devlin, et al. 2023. "Palm: Scaling Language Modeling with Pathways." *Journal of Machine Learning Research* 24 (240): 1–113.

Cobbe, Karl, Vineet Kosaraju, Mohammad Bavarian, et al. 2021. "Training Verifiers to Solve Math Word Problems." *arXiv Preprint arXiv:2110.14168*.

Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. "Bert: Pre-Training of Deep Bidirectional Transformers for Language Understanding." *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4171–86.

Dua, Dheeru, Shivanshu Gupta, Sameer Singh, and Matt Gardner. 2022. "Successive Prompting for Decomposing Complex Questions." *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 1251–65.

Fu, Yao, Hao Peng, Ashish Sabharwal, Peter Clark, and Tushar Khot. 2022. "Complexity-Based Prompting for Multi-Step Reasoning." *arXiv Preprint arXiv:2210.00720*.

Geva, Mor, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. 2021. "Did Aristotle Use a Laptop? A Question Answering Benchmark with Implicit Reasoning Strategies." *Transactions of the Association for Computational Linguistics* 9: 346–61.

He, Junxian, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. 2021. "Towards a Unified View of Parameter-Efficient Transfer Learning." *arXiv Preprint arXiv:2110.04366*.

Hendrycks, Dan, Collin Burns, Saurav Kadavath, et al. 2021. "Measuring Mathematical Problem Solving with the Math Dataset." *arXiv Preprint arXiv:2103.03874*.

Hosseini, Mohammad Javad, Hannaneh Hajishirzi, Oren Etzioni, and Nate Kushman. 2014. "Learning to Solve Arithmetic Word Problems with Verb Categorization." *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 523–33.

Houlsby, Neil, Andrei Giurgiu, Stanislaw Jastrzebski, et al. 2019. "Parameter-Efficient Transfer Learning for NLP." *International Conference on Machine Learning*, 2790–99.

Huang, Jie, and Kevin Chen-Chuan Chang. 2023. "Towards Reasoning in Large Language Models: A Survey." *Findings of the Association for Computational Linguistics: ACL 2023*, 1049–65.

Khot, Tushar, Harsh Trivedi, Matthew Finlayson, et al. 2022. "Decomposed Prompting: A Modular Approach for Solving Complex Tasks." *arXiv Preprint arXiv:2210.02406*.

Kojima, Takeshi, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. "Large Language Models Are Zero-Shot Reasoners." *Advances in Neural Information Processing Systems* 35: 22199–213.

Koncel-Kedziorski, Rik, Hannaneh Hajishirzi, Ashish Sabharwal, Oren Etzioni, and Siena Dumas Ang. 2015. "Parsing Algebraic Word Problems into Equations." *Transactions of the Association for Computational Linguistics* 3: 585–97.

Koncel-Kedziorski, Rik, Subhro Roy, Aida Amini, Nate Kushman, and Hannaneh Hajishirzi. 2016. "MAWPS: A Math Word Problem Repository." *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 1152–57.

Li, Yifei, Zeqi Lin, Shizhuo Zhang, et al. 2023. "Making Language Models Better Reasoners with Step-Aware Verifier." *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 5315–33.

Ling, Wang, Dani Yogatama, Chris Dyer, and Phil Blunsom. 2017. "Program Induction by Rationale Generation: Learning to Solve and Explain Algebraic Word Problems." *Proceedings of the 55th Annual Meeting of the*

Association for Computational Linguistics (Volume 1: Long Papers), 158–67.

Liu, Yinhan, Myle Ott, Naman Goyal, et al. 2019. “Roberta: A Robustly Optimized Bert Pretraining Approach.” *arXiv Preprint arXiv:1907.11692*.

Lu, Pan, Liang Qiu, Kai-Wei Chang, et al. 2022. “Dynamic Prompt Learning via Policy Gradient for Semi-Structured Mathematical Reasoning.” *arXiv Preprint arXiv:2209.14610*.

Patel, Arkil, Satwik Bhattamishra, and Navin Goyal. 2021. “Are NLP Models Really Able to Solve Simple Math Word Problems?” *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2080–94.

Press, Ofir, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. 2023. “Measuring and Narrowing the Compositionality Gap in Language Models.” *Findings of the Association for Computational Linguistics: EMNLP 2023*, 5687–711.

Roy, Subhro, and Dan Roth. 2015. “Solving General Arithmetic Word Problems.” *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, 1743–52.

Saparov, Abulhair, and He He. 2022. “Language Models Are Greedy Reasoners: A Systematic Formal Analysis of Chain-of-Thought.” *arXiv Preprint arXiv:2210.01240*.

Shaikh, Omar, Hongxin Zhang, William Held, Michael Bernstein, and Diyi Yang. 2023. “On Second Thought, Let’s Not Think Step by Step! Bias and Toxicity in Zero-Shot Reasoning.” *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 4454–70.

Sun, Tianxiang, Yunfan Shao, Hong Qian, Xuanjing Huang, and Xipeng Qiu. 2022. “Black-Box Tuning for Language-Model-as-a-Service.” *International Conference on Machine Learning*, 20841–55.

Suzgun, Mirac, Nathan Scales, Nathanael Schärli, et al. 2023. “Challenging Big-Bench Tasks and Whether Chain-of-Thought Can Solve Them.” *Findings of the Association for Computational Linguistics: ACL 2023*, 13003–51.

Talmor, Alon, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. “Commonsenseqa: A Question Answering Challenge Targeting Commonsense Knowledge.” *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4149–58.

Thoppilan, Romal, Daniel De Freitas, Jamie Hall, et al. 2022. “Lamda: Language Models for Dialog Applications.” *arXiv Preprint arXiv:2201.08239*.

Wang, Boshi, Sewon Min, Xiang Deng, et al. 2023. “Towards Understanding Chain-of-Thought Prompting: An Empirical Study of What Matters.” *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2717–39.

Wang, Zihao, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojian Ma, and Yitao Liang. 2023. “Describe, Explain, Plan and Select: Interactive Planning with Large Language Models Enables Open-World Multi-Task Agents.” *arXiv Preprint arXiv:2302.01560*.

Wei, Jason, Yi Tay, Rishi Bommasani, et al. 2022. “Emergent Abilities of Large Language Models.” *arXiv Preprint arXiv:2206.07682*.

Wei, Jason, Xuezhi Wang, Dale Schuurmans, et al. 2022. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.” *Advances in Neural Information Processing Systems* 35: 24824–37.

Weng, Yixuan, Minjun Zhu, Shizhu He, Kang Liu, and Jun Zhao. 2022. “Large Language Models Are Reasoners with Self-Verification.” *arXiv Preprint arXiv:2212.09561* 2.

Yao, Shunyu, Jeffrey Zhao, Dian Yu, et al. 2022. “React: Synergizing Reasoning and Acting in Language Models.” *The Eleventh International Conference on Learning Representations*.

Zhang, Zhuosheng, Aston Zhang, Mu Li, and Alex Smola. 2022. “Automatic Chain of Thought Prompting in Large Language Models.” *arXiv Preprint arXiv:2210.03493*.

Zhou, Denny, Nathanael Schärli, Le Hou, et al. 2022. “Least-to-Most Prompting Enables Complex Reasoning in Large Language Models.” *arXiv Preprint arXiv:2205.10625*.

A 附录

本节包含两个部分：（1）我们尝试过的所有提示词的结果；（2）由 Zero-shot-PS+ 生成的示例文本。除非另有说明，否则我们均使用 GPT-3 (text-davinci-003) 模型。

A.1 所有触发句的结果

表 7 至表 16 列出了我们在每个数据集上尝试过的所有提示词的测试结果。

A.2 Zero-shot-PS+ 的输出示例

表 17 至表 25 列出了 Zero-shot-PS+ 在每个数据集上生成的输出示例。

表 7：在 AQUA 数据集上使用 text-davinci-003 进行 Zero-shot-PS+ 提示步骤 1 中所用提示词的性能比较

编号	触发句	准确率 (%)
1	让我们首先理解问题并制定一个解决该问题的计划。然后，让我们执行该计划，一步一步地解决问题。	42.5
2	让我们首先理解问题，仔细提取所有相关变量及其对应的数字，并制定一个计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），仔细地一步一步解决问题，并展示答案。	42.9
3	让我们首先理解问题，提取相关正确的变量及其对应的正确数字，并制定完整的计划。然后，让我们执行该计划，计算包含所提取变量的中间变量（注意正确的数值计算和常识），通过单方程解决问题，并展示答案。	43.7
4	让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。	46.0

表 8：在 GSM8K 数据集上使用 text-davinci-003 进行 Zero-shot-PS+ 提示步骤 1 中所用提示词的性能比较

编号	触发句	准确率 (%)
1	让我们首先理解问题并制定一个解决该问题的计划。然后，让我们执行该计划，一步一步地解决问题。	58.2
2	让我们首先理解问题，提取相关变量及其对应的数字，并制定一个计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。	58.7
3	让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。	59.3

表 9：在 MultiArith 数据集上使用 text-davinci-003 进行 Zero-shot-PS+ 提示步骤 1 中所用提示词的性能比较

编号	触发句	准确率 (%)
1	让我们首先理解问题并制定一个解决该问题的计划。然后，让我们执行该计划，一步一步地解决问题。	87.2
2	让我们首先理解问题，提取相关变量及其对应的数字，并制定一个计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。	88.3
3	让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意计算的正确性和常识），一步一步地解决问题，并展示答案。	90.5
4	让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。	91.8

表 10：在 SVAMP 数据集上使用 text-davinci-003 进行 Zero-shot-PS+ 提示步骤 1 中所用提示词的性能比较

编号	触发句	准确率 (%)
1	让我们首先理解问题并制定一个解决该问题的计划。然后，让我们执行该计划，一步一步地解决问题。	72.0
2	让我们首先理解问题，提取相关变量及其对应的数字，并制定一个计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。	75.4
3	让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。	75.7

表 11：在 AddSub 数据集上使用 text-davinci-003 进行 Zero-shot-PS+ 提示步骤 1 中所用提示词的性能比较

编号	触发句	准确率 (%)
1	让我们首先理解问题并制定一个解决该问题的计划。然后，让我们执行该计划，一步一步地解决问题。	87.3
2	让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。	87.8
3	让我们首先理解问题，提取相关变量及其对应的数字，并制定一个计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。	92.2

表 11：在 AddSub 数据集上使用 text-davinci-003 进行 Zero-shot-PS+ 提示步骤 1 中所用提示词的性能比较

编号	触发句	准确率 (%)
1	让我们首先理解问题并制定一个解决该问题的计划。然后，让我们执行该计划，一步一步地解决问题。	87.3
2	让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。	87.8
3	让我们首先理解问题，提取相关变量及其对应的数字，并制定一个计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。	92.2

表 12：在 SingleEq 数据集上使用 text-davinci-003 进行 Zero-shot-PS+ 提示步骤 1 中所用提示词的性能比较

编号	触发句	准确率 (%)
1	让我们首先理解问题并制定一个解决该问题的计划。然后，让我们执行该计划，一步一步地解决问题。	92.3
2	让我们首先理解问题，提取相关变量及其对应的数字，并制定一个计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。	94.7

表 13：在 CSQA 数据集上使用 text-davinci-003 进行 Zero-shot-PS+ 提示步骤 1 中所用提示词的性能比较

编号	触发句	准确率 (%)
1	让我们制定一个计划并一步一步地解决问题。	67.4
2	让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。	71.9

表 14：在 StrategyQA 数据集上使用 text-davinci-003 进行 Zero-shot-PS+ 提示步骤 1 中所用提示词的性能比较

编号	触发句	准确率 (%)
1	让我们制定一个计划并一步一步地解决问题。	61.5
2	让我们制定一个完整的计划。然后，让我们执行该计划，一步一步地解决问题，并展示答案。	63.0
3	让我们首先准备相关信息并制定一个计划。然后，让我们一步一步地回答问题（注意常识和逻辑连贯性）。	65.4

表 15：在 Last Letters 数据集上使用 text-davinci-003 进行 Zero-shot-PS+ 提示步骤 1 中所用提示词的性能比较

编号	触发句	准确率 (%)
1	让我们制定一个计划并一步一步地解决问题。	75.2

表 16：在 Coin Flip 数据集上使用 text-davinci-003 进行 Zero-shot-PS+ 提示步骤 1 中所用提示词的性能比较

编号	触发句	准确率 (%)
1	让我们制定一个完整的计划。然后，让我们执行该计划，一步一步地解决问题，并展示答案。	70.6
2	让我们首先制定一个计划，然后一步一步地解决问题。	72.6
3	让我们首先制定一个计划，然后一步一步地解决问题。（区分反面朝上和正面朝上）	84.0
4	让我们首先理解问题并制定一个完整的计划。然后，让我们执行该计划，一步一步地对问题进行推理。每一步回答子问题“这个人翻转了吗，硬币当前的状态是什么？”，并根据硬币的最后状态给出最终答案（注意每次翻转硬币都会改变状态）。	94.2
5	让我们首先理解问题并制定一个完整的计划。然后，让我们执行该计划，一步一步地对问题进行推理。每一步回答子问题“这个人翻转了吗，硬币当前的状态是什么？”，并根据硬币的最后状态给出最终答案（注意每一次翻转，硬币都会改变状态）。	99.0
6	让我们首先理解问题并制定一个完整的计划。然后，让我们执行该计划并一步一步地对问题进行推理。每一步回答子问题，“这个人翻转了吗，硬币当前的状态是什么？”。根据硬币的最后状态，给出最终答案（注意每一次翻转和硬币的翻转状态）。	99.6

表 17：Zero-shot-PS+ 在 AQuA 数据集上的输出示例

问(Q): 一名工人在这两周（包含 15 个连续工作日）内的平均工资是每天 90 美元。在前 7 天，他的平均工资是每天 87 美元，后 7 天的平均工资是每天 92 美元。他第 8 天的工资是多少？选项：(A) 83 美元 (B) 92 美元 (C) 90 美元 (D) 97 美元 (E) 104 美元 答(A): 让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。 输出 (Output): 变量： 前 7 天的平均工资 = 87 后 7 天的平均工资 = 92 这两周的平均工资 = 90 计划： 计算前 7 天赚取的总工资。 计算后 7 天赚取的总工资。 计算这两周内赚取的总工资。 计算第 8 天的工资。 计算： 前 7 天赚取的总工资 = $7 \times 87 = 609$ 后 7 天赚取的总工资 = $7 \times 92 = 644$ 这两周内赚取的总工资 = $15 \times 90 = 1350$ 第 8 天的工资 = $(1350 - 609 - 644) / 1 = 97$ 答案：(D) 97 美元。

问 (Q): 如果 27 瓶苏打水花费 A 美分，那么 B 瓶需要多少美元？选项：(A) AB/2700 (B) 27/AB (C) AB/270 (D) 2700/AB (E) 100AB/27

答 (A): 让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。

输出 (Output):
已知:
27 瓶苏打水花费 A 美分
B 瓶苏打水
计划:
将 A 美分转换为美元
计算 B 瓶苏打水的花费
解答:
将 A 美分转换为美元: A 美分 = $A / 100$ 美元
计算 B 瓶苏打水的花费: B 瓶苏打水 = $(A / 100) \times (B / 27) = AB / 2700$, 美元
答案: (A) AB/2700

表 18: Zero-shot-PS+ 在 GSM8K 数据集上的输出示例

问(Q): James 决定每周跑 3 次冲刺, 每次 3 趟。他每趟冲刺跑 60 米。他一周总共跑多少米?

答(A): 让我们首先理解问题, 提取相关变量及其对应的数字, 并制定一个完整的计划。然后, 让我们执行该计划, 计算中间变量 (注意正确的数值计算和常识), 一步一步地解决问题, 并展示答案。

输出 (Output):
已知:
James 每周跑 3 次冲刺, 每次 3 趟。
每趟冲刺 60 米。
计划:
我们需要计算 James 一周跑的总米数。
计算:
James 一周跑的冲刺总趟数 = 3 趟 $\times$ 次 = 9 趟
James 一周跑的总米数 = 9 趟 $\times$ 60 米 = 540 米
答案: James 一周跑 540 米。

问 (Q): 一个 20 名学生的舞蹈班里, 20% 报名了现代舞, 剩余的 25% 报名了爵士舞, 其余的报名了嘻哈舞。报名嘻哈舞的学生占全体学生的百分比是多少?

答 (A): 让我们首先理解问题, 提取相关变量及其对应的数字, 并制定一个完整的计划。然后, 让我们执行该计划, 计算中间变量 (注意正确的数值计算和常识), 一步一步地解决问题, 并展示答案。

输出 (Output):
已知:
学生总数 = 20
报名现代舞的学生百分比 = 20%
报名爵士舞的学生百分比 = 25%
计划:
计算报名现代舞的学生人数。
计算减去报名现代舞的学生人数后剩余的学生人数。
计算报名爵士舞的学生人数。
计算报名嘻哈舞的学生人数。
计算报名嘻哈舞的学生百分比。
计算:
报名现代舞的学生人数 = 20 的 20% = $20\% \times 20 = 4$
减去报名现代舞的学生人数后剩余的学生人数 = $20 - 4 = 16$
报名爵士舞的学生人数 = 16 的 25% = $25\% \times 16 = 4$
报名嘻哈舞的学生人数 = $16 - 4 = 12$
报名嘻哈舞的学生百分比 = $12 / 20 \times 100\% = 60\%$
答案: 全体学生中有 60% 报名了嘻哈舞。

表 19: Zero-shot-PS+ 在 MultiArith 数据集上的输出示例

问(Q): 学校食堂为学生午餐订购了 42 个红苹果和 7 个青苹果。但是, 如果只有 9 个学生想要水果, 食堂最终会多出多少个苹果?

答(A): 让我们首先理解问题, 提取相关变量及其对应的数字, 并制定一个完整的计划。然后, 让我们执行该计划, 计算中间变量 (注意正确的数值计算和常识), 一步一步地解决问题, 并展示答案。

输出 (Output):
变量:
红苹果 = 42
青苹果 = 7
想要水果的学生 = 9
计划:
计算食堂订购的苹果总数
从食堂订购的苹果总数中减去想要水果的学生人数
计算:
食堂订购的苹果总数 = $42 + 7 = 49$
答案: 多出的苹果 = $49 - 9 = 40$

问 (Q): Carol 和她的妈妈正在从她们的花园里拔胡萝卜。Carol 拔了 29 根, 她的母亲拔了 16 根。如果只有 38 根胡萝卜是好的, 她们有多少根坏胡萝卜?

答 (A): 让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。

输出 (Output):

变量:

Carol: 29

妈妈: 16

好胡萝卜: 38

计划:

我们可以通过从拔出的胡萝卜总数中减去好胡萝卜的数量来计算坏胡萝卜的数量。

计算:

拔出的胡萝卜总数 = Carol + 妈妈 = 29 + 16 = 45

坏胡萝卜 = 拔出的胡萝卜总数 - 好胡萝卜 = 45 - 38 = 7

答案: 有 7 根坏胡萝卜。

表 20: Zero-shot-PS+ 在 SVAMP 数据集上的输出示例

问(Q): 妈妈总共买了 70 件白色 T 恤。如果白色 T 恤可以按包购买，并且妈妈买了 14 包。每包有多少件白色 T 恤？

答(A): 让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。

输出 (Output):

变量:

白色 T 恤总数 = 70

包数 = 14

计划:

我们可以使用方程式:

白色 T 恤总数 = 包数 × 每包的 T 恤数量

计算:

每包的 T 恤数量 = 70 / 14 = 5

答案: 每包有 5 件白色 T 恤。

问 (Q): Jesse 的房间长 19 英尺，宽 18 英尺。如果她总共有 20 个房间。她需要多少地毯来铺满所有房间的地板？

答 (A): 让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。

输出 (Output):

相关变量:

房间数: 20

每个房间的长度: 19 英尺

每个房间的宽度: 18 英尺

计划:

步骤 1: 计算每个房间的面积。

步骤 2: 将每个房间的面积乘以房间数。

步骤 3: 展示答案。

计算:

步骤 1: 每个房间的面积 = 长度 × 宽度 = 19 英尺 × 18 英尺 = 342 平方英尺

步骤 2: 所有房间的总面积 = 每个房间的面积 × 房间数 = 342 平方英尺 × 20 间 = 6840 平方英尺

答案: Jesse 需要 6840 平方英尺的地毯来铺满所有房间的地板。

表 21: Zero-shot-PS+ 在 AddSub 数据集上的输出示例

问(Q): 谷仓里有 28 捆干草。Tim 今天在谷仓里堆了干草。现在谷仓里有 54 捆干草。他在谷仓里存放了多少捆干草？

答(A): 让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。

输出 (Output):

已知:

谷仓里干草的初始数量 = 28

Tim 堆放后谷仓里的干草数量 = 54

计划:

通过从 Tim 堆放后的干草数量中减去初始的干草数量，计算 Tim 在谷仓中存放的干草数量。

计算:

Tim 在谷仓中存放的干草数量 = 54 - 28 = 26

答案: Tim 在谷仓里存放了 26 捆干草。

问 (Q): 花瓶里有 6 朵玫瑰。Mary 从她的花园里剪了一些玫瑰。现在花瓶里有 16 朵玫瑰。她剪了多少朵玫瑰？

答 (A): 让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。

输出 (Output):

已知:

花瓶里玫瑰的初始数量 = 6
花瓶里玫瑰的最终数量 = 16
计划：
从玫瑰的最终数量中减去玫瑰的初始数量，计算出 Mary 剪下的玫瑰数量。
计算：
Mary 剪下的玫瑰数量 = 16 - 6 = 10
答案：Mary 从她的花园里剪了 10 朵玫瑰。

表 22: Zero-shot-PS+ 在 SingleEq 数据集上的输出示例

问(Q): 在餐厅吃完饭后，Sally、Sam 和 Alyssa 决定平摊账单。如果每个人付了 45 美元，账单总额是多少？

答(A): 让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。

输出 (Output):
问题：在餐厅吃完饭后，Sally、Sam 和 Alyssa 决定平摊账单。如果每个人付了 45 美元，账单总额是多少？
变量：
账单总额 (T)
人数 (N)
每个人付的金额 (A)
数字：
N = 3
A = 45
计划：
将人数 (N) 乘以每个人付的金额 (A) 来计算账单总额 (T)。
计算：
T = N × A
T = 3 × 45
T = 135
答案：账单总额是 135 美元。

问 (Q): Mary 在烤蛋糕。食谱上需要 8 杯面粉。她已经放了 2 杯。她还需要加多少杯？

答 (A): 让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。

输出 (Output):
已知：
Mary 需要往蛋糕里加 8 杯面粉。她已经放了 2 杯。
计划：
从需要的面粉总量中减去 Mary 已经放入的面粉量。
计算：
8 杯（需要的面粉总量）- 2 杯（Mary 已经放入的面粉量）= 6 杯（Mary 需要加的面粉量）
答案：Mary 还需要加 6 杯面粉。

表 23: Zero-shot-PS+ 在 CommonsenseQA 数据集上的输出示例

问(Q): 什么是隐匿在树林中的长椅所在的地方？选项：(A) 州立公园 (B) 公交车站 (C) 公交车库 (D) 雕像 (E) 火车站

答(A): 让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。

输出 (Output):
问题在询问一个隐匿在树林中且有长椅的地方。
选项有 (A) 州立公园，(B) 公交车站，(C) 公交车库，(D) 雕像，和 (E) 火车站。
答案是 (A) 州立公园。州立公园通常位于有树木的自然区域，并且通常有长椅供游客坐下欣赏风景。

问 (Q): 在哪里放灭火器是个好主意，但不是强制要求的？选项：(A) 校车 (B) 船 (C) 房子 (D) 医院 (E) 学校

答 (A): 让我们首先理解问题，提取相关变量及其对应的数字，并制定一个完整的计划。然后，让我们执行该计划，计算中间变量（注意正确的数值计算和常识），一步一步地解决问题，并展示答案。

输出 (Output):
问题在询问放灭火器是个好主意但并非强制要求的地方。
选项有 (A) 校车，(B) 船，(C) 房子，(D) 医院，和 (E) 学校。
最好的答案是 (C) 房子。在房子里放灭火器是个好主意，但并非强制要求。
校车、船、医院或学校通常不会不要求配备灭火器（注：原句此处语境意为这些地方通常是强制要求配备的）。

表 24: Zero-shot-PS+ 在 StrategyQA 数据集上的输出示例

问(Q): 在 1976 年奥运会上获得金牌最多的国家如今还存在吗？

答(A): 让我们首先准备相关信息并制定一个计划。然后，让我们一步一步地回答问题（注意常识和逻辑连贯性）。

输出 (Output):

相关信息:

1976 年奥运会在加拿大蒙特利尔举行。

在 1976 年奥运会上获得金牌最多的国家是苏联。

计划:

- 1. 确定苏联是否仍然存在。
- 2. 如果苏联仍然存在，则回答该问题。
- 3. 如果苏联不存在，则确定它被哪些国家所取代。

答案:

- 1. 苏联已经不存在了
- 2. 不，在 1976 年奥运会上获得金牌最多的国家已经不存在了
- 3. 苏联已被 15 个独立国家所取代，其中包括俄罗斯、乌克兰、白俄罗斯和哈萨克斯坦

问 (Q): 蒙娜丽莎和断臂维纳斯在同一个博物馆吗？

答 (A): 让我们首先准备相关信息并制定一个计划。然后，让我们一步一步地回答问题（注意常识和逻辑连贯性）。

输出 (Output):

相关信息:

蒙娜丽莎：位于法国巴黎的卢浮宫博物馆

断臂维纳斯：位于法国巴黎的卢浮宫博物馆

计划:

步骤 1：确认蒙娜丽莎和断臂维纳斯的位置

步骤 2：比较它们的位置

答案:

是的，蒙娜丽莎和断臂维纳斯都位于法国巴黎的卢浮宫博物馆。

表 25: Zero-shot-PS+ 在 Last Letters 数据集上的输出示例

问(Q): 取 "Lino Mariel Aditya Elisabeth" 中每个单词的最后一个字母并将它们拼接起来

答(A): 让我们制定一个计划并一步一步地解决问题。

输出 (Output):

步骤 1：取给定字符串中每个单词的最后一个字母。

Lino: o

Mariel: l

Aditya: a

Elisabeth: h

步骤 2：拼接最后的字母。

答案: olah

问 (Q): 取 "Jacky Socorro Mark Wanda" 中每个单词的最后一个字母并将它们拼接起来

答 (A): 让我们制定一个计划并一步一步地解决问题。

输出 (Output):

步骤 1：取每个单词的最后一个字母。

Jacky: y

Socorro: o

Mark: k

Wanda: a

步骤 2：拼接这些字母。

答案: yoka

表 26: Zero-shot-PS+ 在 Coin Flip 数据集上的输出示例

问(Q): 一枚硬币正面朝上。Walter 没有翻转硬币。Mikayla 没有翻转硬币。Larry 没有翻转硬币。Ryan 没有翻转硬币。硬币仍然是正面朝上吗？注意此处的“翻转”是指“反转”。

答(A): 让我们首先理解问题并制定一个完整的计划。然后，让我们执行该计划，一步一步地对问题进行推理。每一步回答子问题，“这个人翻转了吗，硬币当前的状态是什么？””。根据硬币的最后状态，给出最终答案（注意每一次翻转和硬币的翻转状态）。

输出 (Output):

计划:

步骤 1: Walter 没有翻转硬币。

步骤 2: Mikayla 没有翻转硬币。

步骤 3: Larry 没有翻转硬币。

步骤 4: Ryan 没有翻转硬币。

答案: 是的，硬币仍然正面朝上。

解释:

步骤 1: Walter 没有翻转硬币。因此，硬币仍然正面朝上。

步骤 2: Mikayla 没有翻转硬币。因此，硬币仍然正面朝上。

步骤 3: Larry 没有翻转硬币。因此，硬币仍然正面朝上。

步骤 4: Ryan 没有翻转硬币。因此，硬币仍然正面朝上。

结论: 是的，硬币仍然正面朝上。

问 (Q): 一枚硬币正面朝上。Lorena 没有翻转硬币。Shana 没有翻转硬币。Priscilla 没有翻转硬币。Summer 没有翻转硬币。硬币仍然是正面朝上吗？注意此处的“翻转”是指“反转”。

答 (A): 让我们首先理解问题并制定一个完整的计划。然后，让我们执行该计划，一步一步地对问题进行推理。每一步回答子问题，“这个人翻转了吗，硬币当前的状态是什么？”根据硬币的最后状态，给出最终答案（注意每一次翻转和硬币的翻转状态）。

输出 (Output):

计划:

步骤 1: Lorena 没有翻转硬币。

步骤 2: Shana 没有翻转硬币。

步骤 3: Priscilla 没有翻转硬币。

步骤 4: Summer 没有翻转硬币。

答案:

步骤 1: Lorena 没有翻转硬币。因此，硬币仍然正面朝上。

步骤 2: Shana 没有翻转硬币。因此，硬币仍然正面朝上。

步骤 3: Priscilla 没有翻转硬币。因此，硬币仍然正面朝上。

步骤 4: Summer 没有翻转硬币。因此，硬币仍然正面朝上。

最终答案: 是的，硬币仍然正面朝上。
